

Technical Requirements Document (TRD)

Project Name: PT Mahaga Widya Cita Corporate Website & Portal **Version:** 1.0 **Date:** July 2026

1. System Architecture

The platform is built as a monolithic full-stack application leveraging the Next.js App Router for both the frontend (client/server components) and the backend (API routes & CMS).

- **Frontend Framework:** Next.js 15 (React 19, App Router)
- **CMS Framework:** Payload CMS v3 (Native Next.js integration)
- **Language:** TypeScript
- **Styling:** Vanilla CSS (CSS Modules & Global CSS)

2. Infrastructure & Hosting

- **Application Hosting:** Vercel (recommended for Next.js) or any Node.js compatible platform (Render, Railway).
- **Database:** Supabase (PostgreSQL 15+)
- **Object Storage:** Cloudflare R2 (S3-compatible) for storing media assets managed by Payload CMS.
- **Version Control:** GitHub

3. Database Schema Overview (PostgreSQL)

The database is managed by Payload CMS's `@payloadcms/db-postgres` adapter, which handles migrations and schema generation using Drizzle ORM under the hood.

Core Collections:

- **Users:**
 - `email` (String, unique)
 - `password` (String, hashed)
 - `role` (Enum: 'admin', 'editor', 'member')
- **Articles:**
 - `title` (String)
 - `slug` (String, unique)
 - `content` (Rich Text - Lexical)
 - `category` (Relationship to Categories)
 - `author` (Relationship to Users)

- `publishedAt` (Date)
- **Categories:**
 - `name` (String)
 - `slug` (String, unique)
- **Policy Reviews:**
 - `title` (String)
 - `document` (Upload/File)
 - `summary` (Rich Text)

4. Security Requirements

- **Authentication:** JWT-based authentication handled by Payload CMS.
- **Authorization:** Role-Based Access Control (RBAC) preventing 'members' from accessing the admin dashboard or modifying content.
- **Environment Variables:** Secrets (Database URI, Payload Secret) must be kept out of version control using `.env` files.
- **API Protection:** Payload API routes secured by default; public read access explicitly configured per collection.

5. Performance Optimization

- **Caching:** Extensive use of Next.js Server Components and Data Cache to minimize database queries on public pages.
- **Image Optimization:** Next.js `next/image` component for automatic WebP conversion and responsive sizing.
- **Fonts:** `next/font/google` for self-hosted, optimized font loading with zero layout shift.